⚡ DATABRICKS SPARK SQL CHEAT SHEET

Production Reference: DDL, DML, Liquid Clustering, Delta Lake ACID & Medallion Patterns

SPARK SQL | DBR 15.4+ LTS | PHOTON ENGINE | UNITY CATALOG

🏛️ Unity Catalog 3-Level Namespace & DDL

Governance

```sql
-- 3-Level Hierarchy: catalog.schema.table
CREATE CATALOG IF NOT EXISTS prod_lakehouse;
CREATE SCHEMA IF NOT EXISTS prod_lakehouse.silver_finance
  MANAGED LOCATION 'abfss://silver@storage.dfs.core.windows.net/';
USE CATALOG prod_lakehouse; USE SCHEMA silver_finance;

-- Liquid Clustering (Modern replacement for PARTITION BY)
CREATE TABLE transactions (
  txn_id BIGINT GENERATED ALWAYS AS IDENTITY,
  customer_id STRING,
  amount DECIMAL(18,2),
  txn_date DATE
) USING DELTA CLUSTER BY (txn_date, customer_id);

-- Clones: Zero-copy pointer vs physical backup
CREATE TABLE dev_txns SHALLOW CLONE transactions;
CREATE TABLE dr_backup DEEP CLONE transactions;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Liquid Clustering:** `CLUSTER BY (txn_date, customer_id)` groups data dynamically without rigid Hive directory trees, completely eliminating file fragmentation and partition skew.
- **Identity Sequences:** `BIGINT GENERATED ALWAYS AS IDENTITY` guarantees gapless monotonically increasing primary keys coordinated by the Delta ACID commit log.
- **Managed Storage:** `MANAGED LOCATION` grants Unity Catalog full lifecycle control over underlying ADLS Gen2 blob files; table drop purges storage automatically.
- **Zero-Copy Clone:** `SHALLOW CLONE` creates an instantaneous metadata-only pointer to Parquet data for dev/QA with 0 storage duplication; `DEEP CLONE` copies all physical bytes for DR.

💡 **Architecture Rule:** Liquid clustering replaces Z-Order and static partitions. Cluster keys can be evolved at runtime without costly data restructurings.

🔄 Delta Lake ACID Upsert (MERGE INTO)

DML / Medallion

```sql
MERGE INTO silver.customers AS target
USING bronze.stg_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.is_deleted = true THEN
  DELETE
WHEN MATCHED AND target.hash_diff ≠ source.hash_diff THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, status, created_at, updated_at)
  VALUES (source.customer_id, source.email, source.status,
          current_timestamp(), current_timestamp())
WHEN NOT MATCHED BY SOURCE AND target.status = 'ACTIVE' THEN
  UPDATE SET target.status = 'INACTIVE';
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Match Predicate:** `ON target.customer_id = source.customer_id` uses Delta min/max file statistics to prune non-matching Parquet files prior to shuffle join.
- **Delta Change Detection:** `target.hash_diff ≠ source.hash_diff` prevents rewriting unchanged records, minimizing write amplification and storage I/O.
- **Soft Deletes:** `WHEN NOT MATCHED BY SOURCE` handles orphan target records in a single ACID commit, deprecating inactive rows without separate sweeps.
- **ACID Commit:** Writes new Parquet files with updated records and atomically logs file removals in the Delta transaction log `_delta_log/`.

⚡ **Performance:** Always cluster or Z-Order target tables by the MERGE join key to activate Dynamic File Pruning (DFP).

🛠️ Compaction, Z-Order & Vacuum

FinOps / Performance

```sql
-- Compact small files into ~1GB Parquet target files
OPTIMIZE silver.transactions;

-- Multidimensional Space-Filling Z-Order (Legacy/Static tables)
OPTIMIZE silver.transactions ZORDER BY (customer_id, txn_date);

-- Purge unreferenced historical parquet files older than 7 days
VACUUM silver.transactions RETAIN 168 HOURS;

-- Collect CBO statistics for adaptive join planning
ANALYZE TABLE silver.transactions COMPUTE STATISTICS FOR ALL COLUMNS;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **OPTIMIZE:** Bin-packs fragmented small files generated by streaming or concurrent writes into uniform ~1GB files, slashing cloud storage API read costs.
- **Z-ORDER BY:** Interleaves high-cardinality column bits using Hilbert curves, enabling massive file skipping during query predicate evaluation.
- **VACUUM RETAIN 168 HOURS:** Deletes orphaned physical Parquet files committed before the retention window, reclaiming storage without breaking concurrent queries.
- **ANALYZE TABLE:** Computes column cardinalities, null counts, and size metrics for Catalyst Cost-Based Optimization (CBO) and join reordering.

⚠️ **Vacuum Caution:** Never set retention < 168 hours in production. Vacuuming 0 hours breaks concurrent active readers and invalidates time travel.

⏳ Delta Time Travel, Audit & Instant Rollback

Resilience

```sql
-- Query historical snapshot by version integer
SELECT * FROM silver.transactions VERSION AS OF 14;

-- Query snapshot by UTC timestamp string
SELECT * FROM silver.transactions
TIMESTAMP AS OF '2026-10-15 08:30:00';

-- Instant point-in-time recovery (Disaster Rollback)
RESTORE TABLE silver.transactions TO VERSION AS OF 13;

-- Inspect full commit history & user audit trail
DESCRIBE HISTORY silver.transactions LIMIT 10;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **VERSION / TIMESTAMP AS OF:** Instructs the Delta reader to evaluate transaction log commit entries backward to the exact transaction ID requested.
- **RESTORE TABLE:** Executes instantaneous metadata rollback in under 1 second by creating a new forward commit pointing to prior file manifests.
- **DESCRIBE HISTORY:** Exposes granular operational telemetry including user identities, cluster IDs, job triggers, isolation levels, and write metrics.
- **Storage Safety:** Historical snapshots read preserved older Parquet files directly; no physical backups need to be unpacked or downloaded.

🔒 **Auditability:** Delta time travel is guaranteed up to the VACUUM retention threshold. Use for reproducibility in regulatory reporting.

🔗 Chained CTEs, Window Slicing & Set Operations

Core SQL

```sql
WITH raw_txns AS (
  SELECT customer_id, region, amount,
         DENSE_RANK() OVER(PARTITION BY region ORDER BY amount DESC) AS rk
  FROM silver.transactions WHERE txn_date ≥ current_date() - 30
),
top_spenders AS (
  SELECT customer_id, region, amount FROM raw_txns WHERE rk ≤ 10
)
SELECT * FROM top_spenders
UNION ALL
SELECT customer_id, region, amount FROM silver.manual_overrides
EXCEPT DISTINCT
SELECT customer_id, region, amount FROM silver.fraud_blacklist;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Chained CTEs:** Modularizes complex analytical subqueries; Catalyst inlines CTE expressions into an unified execution plan without materialization.
- **DENSE_RANK():** Assigns consecutive rank integers across partition boundaries without gaps, ideal for top-N ranking per region.
- **UNION ALL:** Appends datasets without triggering an expensive distributed shuffle deduplication stage.
- **EXCEPT DISTINCT:** Computes set difference by executing a left anti-join under the hood, purging fraudulent IDs efficiently.

💡 **Catalyst Optimization:** Predicates applied after CTE definitions (e.g. `rk <= 10`) are automatically pushed down to limit window processing.

🥇 Medallion SQL & QUALIFY Deduplication

Architecture

```sql
-- Silver Cleanse: QUALIFY filter eliminates nested subqueries!
CREATE OR REPLACE TABLE silver.orders_cleansed AS
SELECT
  order_id, customer_id, amount,
  CAST(order_date AS DATE) AS order_date,
  _metadata.file_modification_time AS ingested_at
FROM bronze.orders_raw
WHERE amount > 0 AND order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER(
  PARTITION BY order_id ORDER BY ingested_at DESC
) = 1;

-- Gold Aggregation: Pre-computed dimension metrics
CREATE OR REPLACE TABLE gold.customer_metrics AS
SELECT customer_id, COUNT(*) AS orders, SUM(amount) AS ltv
FROM silver.orders_cleansed GROUP BY customer_id;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **QUALIFY Clause:** Filters windowed calculations directly in the projection, replacing convoluted `SELECT * FROM (SELECT ..., ROW_NUMBER() ...) WHERE rn=1` subqueries.
- **Metadata Columns:** `_metadata.file_modification_time` retrieves file write timestamps directly from cloud storage headers without schema overhead.
- **Idempotent Deduplication:** Partitions by business key and orders by timestamp, ensuring late-arriving updates overwrite older state deterministically.
- **Gold Aggregation:** Pre-aggregates high-cardinality metrics for low-latency BI dashboards and Power BI Direct Lake consumption.

⚡ **Engine Performance:** QUALIFY enables Catalyst to execute top-1 window pruning during physical execution, reducing shuffle data transfer by up to 90%.

⚡ DATABRICKS SPARK SQL CHEAT SHEET

Analytical SQL, Complex Types, Higher-Order Functions, Pivot & Engine Tuning

ADVANCED SQL | ANALYTICAL FUNCTIONS | ROW FILTERS / MASKS

📊 Window Functions & Analytical Framing

Analytics

```sql
SELECT customer_id, txn_date, amount,
  -- Ranking: Row_Number (unique), Dense_Rank (no gaps)
  ROW_NUMBER() OVER(w_acct ORDER BY txn_date DESC) AS rn,
  DENSE_RANK() OVER(w_acct ORDER BY amount DESC) AS rk,
  -- Offsets: Prior & Next transaction amount
  LAG(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS prev_amt,
  LEAD(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS next_amt,
  -- Moving 7-Day Rolling Average (Physical rows)
  AVG(amount) OVER(w_acct ORDER BY txn_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS roll_7d_avg,
  -- Cumulative Year-to-Date Sum (Range)
  SUM(amount) OVER(w_acct ORDER BY txn_date
    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_sum
FROM silver.transactions
WINDOW w_acct AS (PARTITION BY customer_id);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Named Window:** `WINDOW w_acct AS (...)` defines reusable window specifications across multiple expressions, eliminating redundant partition declarations.
- **LAG / LEAD:** Offsets row values in constant time without self-joins; default fallback value handles edge null conditions gracefully.
- **ROWS vs RANGE:** `ROWS BETWEEN` operates on physical row offsets; `RANGE BETWEEN` operates on logical value boundaries (e.g. timestamps/dates).
- **Execution Plan:** Windowing routes through a single `WindowExec` physical operator; sorting occurs locally within partition boundaries.

⚡ **Pro Tip:** If source data is clustered/sorted by the window partition key, Catalyst executes windowing without any network shuffle!

🧩 Complex Types: Structs, Arrays & Maps

Semi-Structured

```sql
-- Struct navigation, Array slicing, Map lookup
SELECT
  payload.user.id AS user_id,             -- struct field
  payload.tags[0] AS primary_tag,          -- array 0-index
  payload.attributes['tier'] AS tier,      -- map lookup
  -- Parse raw JSON string into typed Struct
  from_json(raw_payload, 'id BIGINT, name STRING, items ARRAY<STRING>') AS parsed,
  -- Convert Struct into JSON string
  to_json(named_struct('ts', current_timestamp(), 'code', 200)) AS json_str
FROM bronze.events;

-- Explode Array into relational rows
SELECT order_id, item.sku, item.qty, item.unit_price
FROM silver.orders
LATERAL VIEW EXPLODE(line_items) lv AS item;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Struct & Map Navigation:** Dot syntax (`payload.user.id`) and bracket notation (`attributes['tier']`) access nested schema attributes with zero JSON parsing overhead.
- **from_json():** Parses stringified JSON into strongly-typed Catalyst native representations in C++ Photon; enables column pruning on individual fields.
- **named_struct() & to_json():** Dynamically generates composite structs and encodes them to compressed JSON payloads for egress streaming.
- **LATERAL VIEW EXPLODE:** Unrolls multi-valued array elements into multiple relational rows; use `EXPLODE_OUTER` to retain rows with null or empty arrays.

💡 **Storage Recommendation:** Store semi-structured payloads as native Parquet STRUCTs to achieve column pruning and 10x faster query scans.

⚡ Higher-Order Functions (In-Engine Array Ops)

Photon Vectorized

```sql
SELECT order_id,
  -- TRANSFORM: Map elements without exploding!
  TRANSFORM(line_items, x →
    named_struct('sku', x.sku, 'gross', ROUND(x.price * 1.20, 2))
  ) AS taxed_items,

  -- FILTER: Retain only elements matching condition
  FILTER(line_items, x → x.price > 100.0) AS luxury_items,

  -- EXISTS: Returns true if any element satisfies predicate
  EXISTS(line_items, x → x.sku LIKE 'PROMO-%') AS has_promo,

  -- AGGREGATE: Reduce array to scalar sum
  AGGREGATE(line_items, 0.0, (acc, x) → acc + x.price,
            acc → ROUND(acc, 2)) AS total_order_sum
FROM silver.orders;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **TRANSFORM(arr, x → ...):** Applies inline lambda transformations directly to elements inside nested arrays without exploding to intermediate rows.
- **FILTER(arr, x → ...):** Prunes array elements matching the boolean condition entirely in-memory within CPU cache lines.
- **EXISTS(arr, x → ...):** Performs fast short-circuit boolean evaluation, terminating the scan as soon as the first matching element is detected.
- **AGGREGATE(arr, zero, (acc, x) → ..., finish):** Folds array elements into a scalar summary value, operating identically to functional fold/reduce.

🚀 **Vectorized Speed:** Higher-Order functions run in CPU registers inside Tungsten/Photon C++, avoiding Java object allocations and row explosion shuffles.

🔒 Unity Catalog Row Filters & Column Masks

Security

```sql
-- 1. Column Masking (Mask card PAN unless in finance)
CREATE OR REPLACE FUNCTION mask_card_pan(pan STRING)
RETURN CASE
  WHEN is_account_group_member('finance_compliance') THEN pan
  ELSE CONCAT('****-****-****-', RIGHT(pan, 4))
END;

ALTER TABLE silver.payments ALTER COLUMN card_number SET MASK mask_card_pan;

-- 2. Row Filtering (Multi-Tenant isolation)
CREATE OR REPLACE FUNCTION filter_regional_tenants(region STRING)
RETURN is_account_group_member('global_admin') OR region = current_user();

ALTER TABLE silver.payments SET ROW FILTER filter_regional_tenants ON (region);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **SET MASK:** Enforces deterministic column-level data redaction in Unity Catalog; non-privileged roles view only masked suffixes across all query tools.
- **SET ROW FILTER:** Injects an invisible dynamic SQL predicate into every downstream query plan, guaranteeing multi-tenant data segregation.
- **is_account_group_member():** Checks SCIM/Entra ID directory group memberships in real time during query parsing.
- **Zero-Copy Security:** Security rules execute at the engine level without creating duplicate physical views or filtered tables.

🔒 **Universal Governance:** Row filters and masks apply uniformly across SQL, PySpark, Power BI Direct Lake, and Databricks SQL Warehouses.

🔄 Dynamic Pivot, Unpivot & Grouping Sets

Dimensional Modeling

```sql
-- Dynamic Pivot (Row categories to columnar metrics)
SELECT * FROM (
  SELECT region, product_category, revenue FROM silver.sales
) PIVOT (
  SUM(revenue) FOR product_category IN (
    'Hardware' AS hw, 'Cloud' AS cloud, 'Services' AS svc
  )
);

-- Unpivot (Columnar metrics back to normalized rows)
SELECT region, category, revenue FROM silver.pivoted_sales
UNPIVOT (revenue FOR category IN (hw, cloud, svc));

-- Multi-Dimensional Subtotals with CUBE
SELECT region, product, SUM(revenue) FROM silver.sales
GROUP BY CUBE(region, product);
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **PIVOT:** Aggregates categorical dimension keys into horizontal columns in a single scan without chained conditional statements.
- **UNPIVOT:** Normalizes denormalized spreadsheet-style columns back into transactional rows suitable for dimensional modeling.
- **GROUP BY CUBE:** Computes all 2^N grouping permutations in a single pass: `(region, product)` , `(region)` , `(product)` , and Grand Total `()` .
- **Performance Note:** Explicitly supplying the `IN (...)` list avoids an expensive pre-aggregation scan to discover unique keys.

💡 **CUBE vs ROLLUP:** CUBE generates all combinatorial cross-product subtotals; ROLLUP generates strictly hierarchical rollups.

⚡ Spark SQL Performance Hints & Config Knobs

Tuning & FinOps

```sql
-- Join & Skew Optimization Hints
SELECT /*+ BROADCAST(dim), SKEW('fact', 'cust_id') */
  fact.order_id, fact.amount, dim.category_name
FROM silver.orders fact
JOIN silver.categories dim ON fact.cat_id = dim.cat_id;

-- Production Performance Session Knobs
SET spark.sql.shuffle.partitions = auto;
SET spark.sql.adaptive.enabled = true;
SET spark.sql.files.maxPartitionBytes = 134217728; -- 128 MB
SET spark.databricks.delta.optimizeWrite.enabled = true;
SET spark.databricks.delta.autoCompact.enabled = true;
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **BROADCAST Hint:** Replicates lookup tables (<100MB) to all executors, converting expensive SortMergeJoin to shuffle-free BroadcastHashJoin.
- **SKEW Hint:** Forces Adaptive Query Execution (AQE) to detect and split oversized join partitions on the specified key into smaller sub-tasks.
- **maxPartitionBytes (128MB):** Sets target uncompressed split size when reading Parquet from cloud storage, controlling input task counts.
- **optimizeWrite & autoCompact:** Dynamically coalesces partitions during commits to prevent the small-file problem in high-frequency pipelines.

⚡ **Engine Auto-Tuning:** In DBR 15.4+, `spark.sql.shuffle.partitions = auto` lets AQE dynamically tune shuffle partitions at runtime.